DOCENTE	Paolo Belloni
Unità Formativa	**Sistemi di Virtualizzazione**
Argomento	Scheda 3 – Openstack – Web server con NGINX

Laboratorio OpenStack: NGINX in DMZ e API su rete privata

L'esercitazione parte da macchine virtuali create da immagine QCOW2 già preparata, con Docker installato e immagini container già presenti localmente. L'obiettivo è realizzare un'architettura a più livelli in OpenStack nella quale solo la VM NGINX è esposta verso l'esterno tramite Floating IP, mentre le VM API restano su rete privata e non sono raggiungibili direttamente da Internet.
Immagine:

Architettura finale

```
Internet / Browser / curl
        |
        | Floating IP
        |
        | rete-web / DMZ: 10.10.10.0/24
        |
+----------------------------+
| VM Alpine nginx-bastion    |
| eth0: 10.10.10.x           |
| eth1: 192.168.100.x        |
| Container NGINX            |
+----------------------------+
        |
        | rete-lan privata: 192.168.100.0/24
        |
+-------------------+      +--------------------+
| VM api1           |      | VM api2            |
| 192.168.100.61    |      | 192.168.100.142    |
| Container FastAPI |      | Container FastAPI  |
+-------------------+      +--------------------+
```

La VM nginx-bastion ha due schede di rete: una verso la rete-web/DMZ e una verso la rete-lan privata. Le VM api1 e api2 hanno una sola scheda sulla rete privata. In questo modo le API non sono esposte direttamente verso Internet: l'accesso passa sempre da NGINX.
La rete-web deve essere collegata al router OpenStack, perché è la rete usata dalla VM nginx-bastion per ricevere traffico dall'esterno tramite Floating IP. La rete-lan deve invece rimanere privata e non deve essere collegata direttamente al router OpenStack, perché contiene i backend applicativi.

Prerequisiti

- Una immagine QCOW2 già caricata o disponibile per creare le VM (alpine – user=alpine02, password=Ict2026!).
- Sulle VM API: Docker già installato, codice applicativo già presente oppure immagine Docker già costruita localmente.
- Sulla VM nginx-bastion: Docker disponibile oppure installabile; immagine nginx:latest scaricabile o già presente.
- Una rete pubblica o DMZ collegata al router OpenStack, per esempio rete-web 10.10.10.0/24.

- Una rete privata non esposta verso Internet, per esempio rete-lan 192.168.100.0/24.
- Security Group configurati per permettere HTTP/SSH verso nginx-bastion e traffico applicativo 8000 tra nginx-bastion e le API.

1. Creare o verificare la rete privata rete-lan

In Horizon aprire la sezione Network e creare una nuova rete privata. La rete privata sarà usata solo per la comunicazione tra NGINX e le API. Non deve essere collegata direttamente al router OpenStack esterno, altrimenti le VM API non sarebbero più realmente isolate.

```
Nome rete: rete-lan
Nome subnet: subnet-lan
CIDR: 192.168.100.0/24
Gateway: 192.168.100.1
DHCP: abilitato
DNS: 8.8.8.8, 1.1.1.1
Collegamento al router OpenStack: NO
```

Il gateway della subnet può esistere come indirizzo interno della subnet OpenStack, ma la rete non deve avere un percorso diretto verso Internet. Lo scopo è lasciare le VM backend isolate dalla rete pubblica.

2. Creare o verificare la rete-web collegata al router OpenStack

La rete-web rappresenta la DMZ dell'esercitazione. È la rete collegata al router OpenStack e consente alla VM nginx-bastion di ricevere traffico tramite Floating IP. La VM nginx-bastion deve essere agganciata a questa rete con una delle sue interfacce.

```
Nome rete: rete-web
Esempio CIDR: 10.10.10.0/24
Collegamento al router OpenStack: SÌ
Uso: accesso esterno verso nginx-bastion tramite Floating IP
```

Le VM api1 e api2 non devono essere collegate a rete-web. Solo nginx-bastion deve avere una scheda su rete-web.

3. Creare la VM nginx-bastion dalla QCOW2

Creare una VM chiamata nginx-bastion usando l'immagine QCOW2 preparata. Questa VM rappresenta il punto di ingresso dell'applicazione e deve avere due interfacce di rete.

- Prima interfaccia: rete-web, cioè la rete DMZ collegata al router OpenStack e associabile ad un Floating IP.
- Seconda interfaccia: rete-lan, cioè la rete privata 192.168.100.0/24 usata per raggiungere api1 e api2.

La VM nginx-bastion è quindi una macchina multi-homed: una scheda guarda verso l'esterno controllato, l'altra guarda verso i backend privati.

```
eth0 -> rete-web / DMZ
eth1 -> rete-lan / backend privato
```

Dopo la creazione della VM, entrare via SSH su nginx-bastion. Se la seconda interfaccia è presente ma non ha indirizzo IP, alzarla e richiedere un lease DHCP. Su Alpine Linux i comandi sono:

```
ip addr
ip link set eth1 up
udhcpc -i eth1
ip addr
```

Un risultato atteso è simile al seguente:

```
eth0: 10.10.10.170/24
eth1: 192.168.100.74/24
```

4. Creare le VM api1 e api2 dalla QCOW2

Creare due VM chiamate api1 e api2 usando la QCOW2 preparata. Queste VM devono essere collegate solo alla rete privata rete-lan. Non devono avere Floating IP e non devono avere una scheda sulla rete-web.

```
api1:
  rete: rete-lan
  IP di esempio: 192.168.100.61

api2:
  rete: rete-lan
  IP di esempio: 192.168.100.142
```

Gli indirizzi IP possono essere diversi, in base al DHCP di OpenStack. È importante annotare gli IP effettivi perché saranno usati nella configurazione NGINX.

5. Verificare la connettività tra nginx-bastion e le API

Dalla VM nginx-bastion verificare che le VM api1 e api2 siano raggiungibili sulla rete privata. Usare gli IP reali delle due VM.

```
ping 192.168.100.61
ping 192.168.100.142
```

Se i ping non funzionano, controllare che tutte le VM siano realmente collegate alla stessa rete-lan e che i Security Group consentano almeno ICMP e traffico TCP verso la porta applicativa 8000.

6. Avviare i container FastAPI sulle VM api1 e api2

Le VM api1 e api2 partono da una QCOW2 già preparata. Questo significa che Docker e le immagini applicative sono già presenti a bordo della VM. Non serve Internet per eseguire un container già presente localmente. Internet servirebbe solo per operazioni come apt update, docker pull, git clone o pip install.

Su api1 controllare quali immagini sono disponibili:

```
docker images
```

Se l'immagine dell'applicazione si chiama, ad esempio, fastapi-app, avviare il container pubblicando la porta 8000:

```
docker run -d   --name api1   --restart unless-stopped   -p 8000:8000   fastapi-app
```

Su api2 eseguire lo stesso comando, cambiando il nome del container:

```
docker run -d   --name api2   --restart unless-stopped   -p 8000:8000   fastapi-app
```

Verificare su entrambe le VM che il container sia in esecuzione:

```
docker ps
```

Verificare anche localmente la risposta dell'applicazione:

```
curl http://localhost:8000
```

7. Verificare le API dalla VM nginx-bastion

Prima di configurare NGINX è necessario verificare che nginx-bastion riesca a contattare direttamente le API sulla rete privata. Sulla VM nginx-bastion eseguire:

```
curl http://192.168.100.61:8000
curl http://192.168.100.142:8000
```

Se curl non è disponibile sulla VM Alpine, usare wget:

```
wget -qO- http://192.168.100.61:8000
wget -qO- http://192.168.100.142:8000
```

Se entrambe le richieste rispondono, la parte backend è corretta. Se una delle due non risponde, controllare sul relativo backend che il container sia in esecuzione, che la porta 8000 sia pubblicata e che il Security Group consenta il traffico TCP 8000 dalla rete-lan.

8. Preparare la configurazione NGINX sulla VM nginx-bastion

Sulla VM nginx-bastion creare una directory dedicata alla configurazione del container NGINX. Il file nginx.conf verrà montato dentro il container come configurazione principale.

```
mkdir -p /opt/nginx
cd /opt/nginx
```

Creare il file nginx.conf con due backend, uno per api1 e uno per api2:

```
cat > nginx.conf <<'EOF'
events {}

http {

    upstream backend {
        server 192.168.100.61:8000;
        server 192.168.100.142:8000;
    }

    server {
```

```
        listen 80;

        location / {
            proxy_pass http://backend;

            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }
    }
}
EOF
```

La sezione upstream definisce il gruppo di backend verso cui NGINX invierà le richieste. Se non viene specificato un algoritmo diverso, NGINX distribuisce le richieste tra i server con comportamento round-robin.

9. Scaricare o verificare l'immagine NGINX

Se la VM nginx-bastion ha accesso a Internet o l'immagine è già presente, verificare con:

```
docker images
```

Se l'immagine nginx non è presente e la VM può uscire verso Internet, scaricarla con:

```
docker pull nginx:latest
```

Se la VM non ha Internet, l'immagine nginx deve essere stata precaricata nella QCOW2 oppure importata manualmente in Docker. In quel caso docker images deve mostrare nginx:latest o un tag equivalente.

10. Avviare il container NGINX

Avviare NGINX come container esponendo la porta 80 della VM e montando il file di configurazione creato nel passaggio precedente:

```
docker run -d   --name nginx   --restart unless-stopped   -p 80:80   -v
/opt/nginx/nginx.conf:/etc/nginx/nginx.conf:ro   nginx:latest
```

Verificare che il container sia attivo:

```
docker ps
```

Controllare eventuali errori di configurazione o di avvio:

```
docker logs nginx
```

11. Verificare NGINX localmente con curl

Il primo test serve a controllare che il container NGINX stia ascoltando correttamente sulla porta 80 della VM nginx-bastion. Dalla VM nginx-bastion eseguire:

```
curl http://127.0.0.1
```

È possibile anche testare l'indirizzo della scheda DMZ della VM, ad esempio:

```
curl http://10.10.10.170
```

Se la risposta arriva da una delle due API, NGINX sta funzionando come reverse proxy. In caso di errore 502 Bad Gateway, controllare che api1 e api2 siano raggiungibili da nginx-bastion sulla porta 8000.

12. Associare il Floating IP e testare dal browser

Associare un Floating IP alla VM nginx-bastion, sulla scheda collegata alla rete-web. Il Floating IP deve puntare alla VM NGINX, non alle VM api1 o api2. Le API devono restare private.
Dal browser del proprio PC aprire:

```
http://<FLOATING_IP_NGINX>
```

In alternativa, da una shell esterna, eseguire:

```
curl http://<FLOATING_IP_NGINX>
```

Questo è il test più importante dell'intera esercitazione. Se compare la risposta JSON dell'applicazione, significa che il client esterno raggiunge il Floating IP, il Floating IP arriva alla VM nginx-bastion, il container NGINX riceve la richiesta e inoltra il traffico verso api1 o api2 sulla rete privata.

```
Client esterno / Browser
        |
Floating IP
        |
VM nginx-bastion
        |
Container NGINX
        |
rete-lan privata
        |
api1 / api2
```

13. Verificare il bilanciamento tra api1 e api2

Per verificare visivamente che il bilanciamento funzioni, conviene fare in modo che api1 e api2 restituiscano risposte diverse. Per esempio, api1 può restituire server=api1 e api2 può restituire server=api2. Poi, dalla VM nginx-bastion o da un client esterno, eseguire più richieste consecutive:

```
for i in $(seq 1 10); do
  curl -s http://127.0.0.1
  echo
done
```

Oppure, dopo l'associazione del Floating IP, da un client esterno:

```
for i in $(seq 1 10); do
  curl -s http://<FLOATING_IP_NGINX>
  echo
done
```

Se le risposte si alternano tra api1 e api2, il load balancing NGINX sta funzionando correttamente.

14. Security Group consigliati

Per nginx-bastion consentire l'accesso SSH e HTTP/HTTPS dall'esterno. In un laboratorio si può usare 0.0.0.0/0; in un ambiente reale è preferibile limitare SSH al solo IP del docente o della rete autorizzata.

```
nginx-bastion:
  Ingress TCP 22  da 0.0.0.0/0
  Ingress TCP 80  da 0.0.0.0/0
  Ingress TCP 443 da 0.0.0.0/0
  Egress any
```

Per api1 e api2 consentire la porta 8000 solo dalla rete privata o dal Security Group di nginx-bastion. Le API non devono avere Floating IP.

```
api1/api2:
  Ingress TCP 8000 da rete-lan o da nginx-bastion
  Ingress TCP 22 solo da nginx-bastion, se si vuole usare la VM NGINX come bastion SSH
  Egress secondo necessità
```

15. Comandi utili di troubleshooting

Se NGINX non risponde, controllare prima che il container sia acceso:

```
docker ps
```

Se il container non è avviato o si chiude subito, leggere i log:

```
docker logs nginx
```

Se NGINX risponde con 502, verificare dalla VM nginx-bastion che i backend siano raggiungibili:

```
curl http://192.168.100.61:8000
curl http://192.168.100.142:8000
```

Se curl non è disponibile, usare wget:

```
wget -qO- http://192.168.100.61:8000
wget -qO- http://192.168.100.142:8000
```

Se i backend non rispondono, controllare sulle VM api1 e api2 che Docker stia eseguendo i container:

```
docker ps
```

Se la seconda scheda di rete della VM nginx-bastion non ha IP su Alpine, alzarla e richiedere DHCP:

```
ip link set eth1 up
udhcpc -i eth1
ip addr
```

16. Risultato finale

Al termine dell'esercitazione sara' disponibile una infrastruttura nella quale solo nginx-bastion è esposto verso l'esterno. Le applicazioni FastAPI restano su rete privata e sono raggiunte solo tramite NGINX. Questo modello riproduce una architettura tipica DMZ/backend: reverse proxy davanti, applicazioni dietro, accesso pubblico controllato.

```
Client esterno
    |
```

```
Floating IP
    |
nginx-bastion
    |
rete-lan privata
    |
api1 / api2
```